

pa7uiclf

Ciência da Computação

**Análise de Algoritmos e Paradigmas de Linguagens
de Programação**

15/08/2025

Paradigmas de Programação

Conceitos e estrutura de Paradigmas de Programação e Análise de Algoritmos.

Paradigmas de Programação

Objetivos da Aula

- Compreender o conceito de **paradigma de programação**.
- Identificar como diferentes paradigmas moldam a forma de **pensar e estruturar** um programa.
- Comparar **Orientação a Objetos, Programação Lógica e Programação Funcional**.
- Exercitar a tradução de um mesmo problema para diferentes paradigmas.
- Estrutura de dados dos exercícios.

Paradigmas de Programação

1. Introdução

- **Paradigma de programação:**

É o modelo conceitual que guia como programas são **estruturados** e **executados**.

- Ele influencia:

- Como você **pensa o problema**.
- Como você **organiza o código**.
- Como **dados e operações** são relacionados.

Programação Orientada a Objetos

Analogia rápida:

- Paradigma é como uma *lente* ou *mapa mental*.

Imagine querer atravessar um rio:

o **OO**: você cria barcos (objetos) e define como eles interagem. o **Lógico**: você descreve a margem de origem, a margem de destino e as regras de travessia, deixando o sistema encontrar a solução. o **Funcional**: você define funções puras que recebem o estado atual e retornam o próximo estado, até alcançar o destino.

Programação Orientada a Objetos

2. Paradigma Orientado a Objetos (OO)

- **Visão:** O programa é uma **coleção de objetos** que interagem.
- **Objeto** = dados + comportamento.
- Estrutura básica:
 - **Classes** (molde para objetos).
 - **Métodos** (ações dos objetos).
 - **Atributos** (características).

Programação Orientada a Objetos

LINK Codificação: <https://trinket.io/java/7e5125046ae4>

Pensamento 00: "Quais entidades existem? Como elas interagem?"

3. Paradigma Lógico

- **Visão:** programa é um **conjunto de fatos e regras** (predicados) e uma **meta** (objetivo) a ser provada.
- **Base:** lógica matemática (Prolog, por exemplo).
- Você **não diz como** fazer, mas **o que é verdadeiro** e deixa o motor lógico deduzir.
- **Exemplo** em Prolog:
https://drive.google.com/file/d/1_pR0GclfmwhupJBCCPXK7GcRc76Cdm9_/view?usp=drive_link
- **Pensamento lógico:** "Quais são as relações e regras? O sistema encontra o caminho."

4. Paradigma Funcional

- **Visão:** O programa é uma expressão que deve ser avaliada.
- **Funções puras:** sem efeitos colaterais, mesma saída para a mesma entrada.
- Muito usado em Haskell, Lisp, Elixir, mas também suportado em Python, JavaScript e Java.
- Exemplo em Java:

Link: <https://trinket.io/java/b3e5bdd8a396>

5. Comparação Visual

<u>Paradigma</u>	<u>Como vê o programa</u>	<u>Foco principal</u>	<u>Exemplo de Linguagens</u>
Orientado a Objetos	Conjunto de objetos que interagem	Entidades e interações	Java, Python, C++
Lógico	Fatos, regras e metas	Relações e inferência	Prolog
Funcional	Expressões e funções puras	Transformações de dados	Haskell, Lisp, Elixir, Java, Python

Exercícios

6. Atividade Prática em Sala

- **Problema:** Calcular a soma dos quadrados de uma lista de números.
- Implementar **nos três paradigmas**:
 1. OO (Java).
 2. Lógico (Prolog/Portugol - <https://portugol.dev>).
 3. Funcional (Java).

Depois, comparar:

- Qual foi mais fácil de pensar?
- Qual ficou mais legível?
- Qual exigiu mais código?

Resumo: Estruturas de Dados no Código Java

Principais Estruturas Utilizadas:

1. ArrayList

- **Uso:** Armazenar números pares e quadrados
- **Características:** Array dinâmico, acesso $O(1)$, inserção $O(1)$

2. List Interface

- **Uso:** Declaração de listas (via `Arrays.asList()`)
- **Características:** Interface para coleções ordenadas

3. Stream/IntStream

- **Uso:** Processamento funcional dos dados
- **Características:** Pipeline de operações, lazy evaluation

Resumo: Estruturas de Dados no Código Java

Principais Estruturas Utilizadas:

4. StringBuilder

- **Uso:** Construção eficiente de strings para demonstração
- **Características:** Buffer mutável, evita criação múltipla de objetos

Fluxo das Estruturas:

List original → Stream → filtro → IntStream → resultado int



ArrayList (pares) → ArrayList (quadrados) → soma

Por que essas estruturas?

Resumo: Estruturas de Dados no Código Java

Por que essas estruturas?

- **ArrayList**: Flexibilidade para crescimento dinâmico
- **Stream**: Código mais limpo e funcional
- **StringBuilder**: Performance na concatenação de strings
- **List Interface**: Abstração e flexibilidade de implementação

Resumo: Estruturas de Dados no Código Java

Por que essas estruturas?

- **ArrayList**: Flexibilidade para crescimento dinâmico
- **Stream**: Código mais limpo e funcional
- **StringBuilder**: Performance na concatenação de strings
- **List Interface**: Abstração e flexibilidade de implementação